Network Influence Stuff

Antoni Zyla (u5527350) - Supervised by Nathan Griffiths

09/04/2026

Contents

1 Abstract

Influence maximisation is the process of choosing a subset of population within a network to maximise the spread of an idea or concept under a certain model of propogation. Computing the optimal solution is NP hard and therefore not feasible to compute for networks of a practical size. Existing approaches including the greedy algorithm and it's optimisation produce a good theoretical guarantee as an approximation of the optimal solution but tend to scale poorly even on small networks.

In this paper we propose and evaluate two approaches to efficiently solving this problem across three primary models of propagation, the independent cascade model, the decreasing cascade model and the linear threshold model. We test across four (4) real world networks of varying topologies and scale, sampled using the Metropolis Hastings Random Walk with Delayed Acceptance (MHDA) algorithm, a performant and topology preserving method. Statistical analysis is performed using Welch's T Test and a linear mixed effect model to account for correlations between runs on the same network.

Our proposed heuristic investigate locally computable node metrics in conjunction with network wide mesaures and metrics. Performance of an approach is highly dependent upon different network topologies, starting influential set sizes and the underlying community structures of networks. Our strongest performing heuristic is a combination of the degree of a node and the mean degree across it's neighborhood, it consistently matches or exceeds CELF across all of the networks under the decreasing cascade model and the linear threshold model at seed sizes of 50 or greater, for a 1000 seed network sample. The ability to be computed with no global knowledge of the network makes it a performant option which can scale across vast networks.

We also investigate community based methods, attempting to exploit the properties of the greedy modularity maximisation community detection algorithm to capture influential nodes in an efficient manner.

Our results suggest that simple and carefully constructed heuristics can reliably match the performance of a computationally heavy approximation method across a range of real world network topologies. We find community based methods perform very well as shown in literature but there we are unable to make gains past that.

2 Introduction

If we were a marketer with access to a population of people and some information about the relationships between them, we may choose a subset of this population who would most effectively market our product. We can consider a population of people in a graph as nodes with relationships between them as edges. Using a model of conceptual contagion we can choose a subset of nodes to act as seeds from which ideas propogate, they may have a direct chance of propagating to their neighbors or another non direct effect.

Choosing an optimal subset of the population that will produce the greatest influence spread is an NP hard problem via a reduction from set cover. Approaches such as the Greedy algorithm and it's optimisation CELF (Cost Effective Lazy Forward) produce a very good theoretical approximation of $(1 - 1/e)$ of the optimal result (this is approximately 63%). Due to the random nature of activations based on a probability p using these greedy based approaches takes a For a set trials of the greedy algorithm over a range of seed sizes on our networks it takes around (TODO) hours, the same for CELF takes approximately (TODO) minutes with simpler heuristics taking just (TODO).

Real world networks may have tens or even hundreds of thousands of nodes with millions of edges between them. Even CELF may be far too slow to operate on such a scale despite being approximately 700 times faster than a naive greedy approach. Approaches requiring knowledge of the entire network to compute some measure about a node may be unfeasible to compute on such networks. Finding lightweight approaches is therefore crucial to efficient influence maximisation in larger networks. Degree based methods have been shown to be highly effective across a range of networks as they exploit the scale free property of typical social networks in which the majority of influence is held by few nodes. Community based approaches aim to exploit the typical clustering of nodes into dense communities with strong inter community links but poor intra community connectivity.

We find that a single lightweight heuristic, made by giving each node a score of the ratio of their degree to the mean degree of their immediate neighbourhood is able to outperform CELF across each network topology under the decreasing cascade model and the linear threshold model. We also investigate methods used in community detection or partitioning of networks to find influential nodes.

3 Literature Review

3.1 What is the Influence Maximisation Problem

Definition: Influence Maximisation Problem Given a network $G = (V, E)$, a non negative integer k, and a model of propagation M.

- Find the subset $S \subset V$ of size k such that the expected spread through G under the model M with S as the starting seeds is maximised.

3.2 Why is it NP-Hard?

Under a given influence influence model choosing the optimal starting set is an NP-hard problem. We can prove this using a reduction from a known NP hard problem, Set Cover.

Definition: Set Cover Given a universe of elements U, a set of subsets of U, $S = \{S_1, S_2, ..., S_m\}$, and a non negative integer k.

- Return True if there exists $S' \subset S$ where $|S'| = k$ and $\cup S' = U$
- Return False otherwise

Let our model of influence have a probability of 1 of any node influencing another node with a limit on only infecting one hop away. From an instance of set cover we can construct a bipartite graph of two sets of nodes A and B with A made up of the nodes $1, 2, ..m$ with B being each of the elements in the universe. Edges between A and B are constructed from each of the subsets in S. We can solve this transformed instance of set cover by choosing k nodes as the seed which will produce the a resultant influence spread which spans the whole graph. This is the problem of choosing maximally influential set.

We can solve set cover using a simplified version of our problem. In actuality our activation probabilities are able to be fractional, there is no limit on how far the influence of a single hop will penetrate.

3.3 Models of Influence Propagation

We will be focusing on three (3) main methods for influence spread across networks.

1. Independent Cascade Model (ICC) - Each node can take a state out of Active, Inactive and uninfected. Each active node has a probability p to infect each of it's uninfected neighbors. Once a node has been infected this process starts again to all of it's neighbors, the node is in the active state once it is infected [**?**]. Once a node has attempted to infect all of it's neighbors it becomes an inactive node. This is the simplest of the models but also mimics real life well, whether it's the spread of a new software technology or a new restaurant becoming more popular.

2. Decreasing Cascade Model (DCC) - A modification of the ICC model where every attempt of a node to be influenced decreases the probability of it being influenced or infected [**?**]. This could be analogous to a common experience, overreaching of advertisements or correspondence, such as marketing emails which have the opposite to intended effect due to annoyance.

3. Linear Threshold Model (LTM) - Every node v is given a probability of being turned p which is uniformly distributed across the interval $[0, 1]$. When the proportion of node v's neighbors which are infected is greater than or equal to p, v itself becomes infected [**?**]. This models well another common experience, a group reaching a common view in regards to something, whether it's the adoption of a new game, or even a software tool that becomes the 'industry standard' this principle is at play.

Various papers have described modifications of each of these models in a multitude of ways, multiple concepts can be spread wherein they can inhibit the spread of eachother by allowing a node to only adopt a single concept [**?**, **?**]. Vaccination or heard immunity can also be modelled in this manner [**?**].

There are other models which work in different manners, borrowing from biological models such as SIR (Suscptible-Infected-Recovered) for modelling diseases more closely [**?**]. These are somewhat able to be modelled using the cascade models however full parity is not possible. An example is going from infected uninfected is not a behavior that is considered with the cascades and would possibly have large effects on the effectiveness of certain seed selection strategies. It is possible to take an agent based approach to influence giving each agent certain characteristics as used in [**?**] for modelling behavioral risk factors of cardiovascular disease spreading across city scale populations.

Developed in 2008 the heat diffusion model (HDM) works on a similar principle to the linear threshold model to model the spread of influence in social networks as accurately as possible. The network has a parameter α which controls the rate at which heat flows between nodes, each node is given a threshold θ which is the minimum 'heat' required for it to become active. The model also has a limit on how long it will run for a period of time t. The rate of heat flow between two nodes is determined by the difference in heat and the value of α, the more nodes are connected to a node spreading heat faster it will be conducted.

Social networks, communication systems, power grids, logistics systems are constantly evolving systems. Nodes are constantly added or removed, connections between nodes are frequenrly added, removed or rerouted, working on a snapshot of an ever evolving network may not be the most effective method for real world problem solving. Dynamic models of influence are models where paths between nodes can be added or removed as part of the process [**?**], this can for example be used to model the evolution of social networks, and how influential nodes may become more or less influential based on different factors or topology, rather than studying networks which are at a fixed point in their evolution. Influence can be studied over these evolutions, however this is an additional large jump in complexity and scope.

Another way in which this field has evolved is the study of adversarial contagions. Analogous to the spread of a disease against a vaccination, two concepts/contagion spreading across the network at the same time, each node can only be infected with one and may or may not be turned from one to the other. (SOURCE)

3.4 Working with large networks

Sampling networks is a way of preserving the topological features and general properties of a network whilst giving smaller networks to allow for easier computation. This allows us to create strategies by quickly testing on small networks to what works and what doesn't and then due to being a representative sample we know that they will generalise well to larger networks.

3.4.1 Naive Approaches

A simple natural approach could be to take a node at random from the graph, call it v, and then perform a breadth first search in layers across the graph. An issue with naive BFS is that when a high degree node is encountered then each of it's neighbours would also be added, this will inflate the average degree of the network by tending to sample a subset with a higher mean degree [**?**].

Working outwards from a start node does not preserve the distribution of shortest path distances between nodes or clustering coefficients well. Nodes close to the root will maintain accurate values but as we move further and further out a node is less likely to appear before we have no more nodes needed leading to a lower shorter mean path length and a higher clustering coefficient over other methods [**?**]. Some papers describe BFS as adding the entire layer from which it explores, leading to oversized samples [**?**]. Taking a BFS sample of 1000 nodes in the Enron dataset can

produce a subgraph of over 12 thousand nodes which is not comparable to a standard 1000 node sample.

It is important to compare like for like network sizes, topological features such as diameter, maximum degree or mean shortest path length are closely dependant on overall network size.

Snowball sampling is a variation on breadth first search, once the addition of the final layer would take the total number of sampled nodes over the sampling threshold, nodes are added one at a time until the exact number of nodes is reached [**?**].

A random walk is another simple strategy for exploring a graph. Working on the basis of a standard random walk, starting from a node v there is a $\frac{1}{deg(v)}$ probability of choosing the next node to be u where u is one of the neighbours of v. A random walk based approach will suffer from the same problems as BFS with finding higher degree nodes much more often than lower degrees [**?**], as a higher degree node has more connections that lead to it.

3.4.2 Smarter Sampling Methods

Utilising modified random walks is a standard method of achieving accurate and unbiased samples of large networks such as facebook [**?**]. Metropolis Hastings Random Walks and Random Walks with Delayed Acceptance (MHRW and MHDA) are two common methods used [**?**, **?**]

1. Metropolis Hastings Random Walk (MHRW) - Sample a random uniform probability in the range 0 to 1 and call it p. If for a given node $p \leq$ min
2. Metropolis Hastings Random Walk with Delayed Acceptance (MHDA). This is a further modification of MHRW where nodes with a high degree are less likely to be chosen. Overall producing a more representative sample of the network than MHRW with little extra performance cost.
3. Albatross Sampling (AS) - A random walk based sample with a probability of jumping to a random node within the network, this should allow for better capture of networks with disconnected or densely clustered components or communities on a very large scale.

3.4.3 Scale Free (or not) Networks

A scale free network is one where the distribution of individual node degrees could be modelled using an exponential function. In this case the probability of a node having a degree k is modelled by the below equation.

$$P(k) \sim k^{-\gamma} \tag{1}$$

There exist few nodes with a very high number of connections with the majority of nodes having a low number of connections. Some examples of scale free networks are typical human social networks or even wikipedia articles. A simple way to visualise this is by looking at your facebook or instagram account, you and the majority of people may on average have maybe 20-50 friends or connections, checking the numbers for any actor or influential figure may yield values in the millions.

If a large proportion of the network is able to quickly reached by these very high degree nodes then this informs that the value of a degree based heuristic may prove to be more valuable than any other heuristic. In contrast networks that do not exhibit this behaviour are often not as easily spread through using basic heuristics. Utilising community based heuristics or more complex methods may be more worthwhile for such networks.

3.5 Finding seed nodes

There are a few methods which don't rely on a heuristic to determine the seed nodes.

1. Brute Force, for a graph made up of n nodes there are n choose k possible sets of starting seeds. Running a cascade for each of these would be incredibly inefficient and impractical by being on the order of n factorial. I have not been able to find reference of this being used to compare with other methods likely due to the required processing power.

2. Greedy, starting with an empty seed set choose the node in the subgraph of the network bar our seeds which will produce the largest influence, add this node to our set of seeds and continue to add nodes until we have a set of our desired size. This achieves a $1 - 1/e$ approximation ratio to the optimal solution due to exploiting the sub-modularity of additional nodes being added to the overall spread of ideas.

3. Cost Effective Lazy Forward (CELF). A modification of the greedy algorithm to prevent recomputation of the expected gain for each node by keeping a sorted heap of the marginal gain for each un-included node and only recomputing when required.

Other factors which can help us gauge how influential a node may be with no global knowledge of the graph required are:

1. Degree (Deg). In the ICC and DCC models a node has some chance p of spreading a concept to it's neighbours it is clear that on average it would spread to an expected $p \deg(v)$ nodes. Likewise on threshold based models a high degree node being chosen will increase the proportion of infected nodes for it's very high number of neighbours triggering a likely large cascade.

2. Degree Discount (DD). It's known that high degree nodes tend to cluster together, choosing multiple tightly coupled high degree nodes is redundant and not as effective as it could be. Removing any neighbours of a selected node and then using degree is an effective way of minimising the overlap of expected influence between chosen seed nodes. Requiring a factor of Δ more computation power than raw degree heuristic it is a performant option. There is a limitation with regards to networks which have a few very high degree nodes, by choosing just a few of these nodes there are often minimal nodes left to choose from and an appropriate sample is not able to obtained.

3. Mean Neighbour Degree (MND). Thinking about how the cascade will propogate, choosing nodes who's neighbours are very high degree nodes will result in gaining access to multiple influential nodes (those with high degree) from a single seed. Compared to raw degree this requires a factor of Δ^2 more computation. For most graphs this is still a very performant option as those very high degree nodes are very rare.

Those above are quick to compute and importantly required no global knowledge of the network.

1. Local Clustering Coefficient (LCC). Consider a node where each of it's neighbours is connected to every other neighbour, this would be a clique and a very highly clustered node. We are measuring how close a node is to being in a clique from it's neighbours by taking the proportion of neighbours connected to other neighbours from the total possible number. Computing this is relatively fast being just a factor of (TODO) more than a degree based metric. Performance is dependant upon network topology. On a very simple level a highly clustered node will perform well at spreading concepts, each of it's neighbours will also be highly connected and help spread the concepts further. There are some cases where it may not be of use, if in a graph there exists a small almost unconnected clique of nodes of size greater than our seed size then choosing them all will not produce a good spread in the rest of the graph.

$$C(v) = \frac{2T(v)}{\deg(v)(\deg(v) - 1)} \tag{2}$$

   (TODO)

2. Betweenness Centrality (BC). The proportion of shortest paths from any two nodes in the network which pass through the point. Take a graph of two communities with a single edge between them, the two nodes on either side of the edge would have a very high betweenness

as they are vital for the connection of the two communities. Most topologies will not exhibit this extreme a distribution in betweenness however it is still a useful metric for influence. As we need to compute the shortest path between each pair of nodes there is a running time on the order of $O(nm)$ (we can cache TODO). Localised approximations for betweenness have been show to be highly correlated with betweenness but only in certain network topologies.

$$B(v) = \sum_{s,t \in V, s \neq v \neq v} \frac{|\sigma(s,t|v)|}{|\sigma(s,t)|} \tag{3}$$

where $\sigma(s,t)$ denotes the set of shortest paths from node s to t, with the conditional on v is shortest paths which include v. h betweenness is only taking into account shortest paths of length less than or equal to h, as h approaches the diameter of the network it becomes a slower but more accurate approximation.

$$B_h(v) = \sum_{s,t \in V, s \neq v \neq t} \frac{\{p|p \in \sigma(s,t|v), |p| \leq h\}}{\{p|p \in \sigma(s,t), |p| \leq h\}} \tag{4}$$

3.6 Community Based Methods

Social networks have a natural community structure whether it's families, workplaces or schools it is feasible to partition a network into a set of communities which span the whole network. Nodes within a community are strongly connected with links between communities being minimal (source). Working on the idea that if a community is isolated from the rest of the network then determining the expected influence of a node within it's community is a good indicator of it's global influence.

Though not working on the diffusion models we are targeting, instead on the heat diffusion model (HDM), Introduced in 2015 by Chen et al Community-Based Influence Maximization (CIM) model worked in three phases, partitioning networks, determining nodes which should be used to select seeds and then choosing the exact selection of seed nodes. To produce a set nodes from which to begin selecting seed nodes from the concept of significant communities was introduced. Significant communities are those where the number of nodes is greater than the mean number of nodes a single node is expected to influence. Pruning the insignificant communities narrows down the number of nodes from which to choose seeds from.

CIM works in three phases

1. Partition the network into communities using the h_clustering algorithm. Nodes can be identified as hubs or outliers.

2. Generate a set of nodes from which to choose our seeds. Considering only nodes which are hubs (connecting two communities), High degree and high similarity nodes within significant (large enough) communities. It was found that just 10 percent of each significant community is enough to get worthwhile seed nodes.

3. Seeds are chosen from each community and split up based on the size of the community to allocate more seeds to larger communities.

Although it works

$$Q = \frac{1}{(2m)} \sum_{vw} \left[A_{vw} - \frac{deg_v deg_w}{(2m)} \right] \delta(c_v, c_w) = \sum_{i=1}^{c} (e_{ii} - a_i^2) \tag{5}$$

where e_{ij} is the proportion of edges connecting communities i and j, a_i is the fraction of ends of edges which end in community i. What is the null model in this scenario (TODO) - assuming a random graph where each vertex has a random probability of being connected to any other vertex. What we are finding via Q is how much different the proportion of edges between communities vs inside communities is in our network to the expectation of a random graph that has the same number of nodes and edges as our graph.

If a network displays a strong community structure then that implies there exist subsets where it is hard for ideas to propogate between them. If you were to choose nodes in just a single community then you might be able to saturate a community with a concept, however this may not provide an optimal global spread of the concept in the network.

In certain scenarios this behaviour may be a good thing, if a community structure was distinct in real world characteristic, maybe consumer spending power or education levels then to propogate ideas would require different approaches and we could model this in a more fine tuned manner.

3.6.1 Community Detection Algorithms

Talk about top down and bottom up methods - greedy modularity and girvan newman greedy - talk about effects of chosoing a different algorithm for ranking the removal/combination of edges.

Introduced in 2006 by MEJ Newman (SOURCE TODO) it was found that there exists an $O(n^2 + nm)$ algorithm for community detection in networks based on the notion of maximizing modularity using a greedy method. Starting from a spectral method (vectors and matrices???) to very roughly partition the network into a set of communities. From this find vertices which will have the greatest gain in modularity for the whole graph, continuing to change the community each vertex belongs to until you cannot increase the modularity of the graph further.

Girvan-Newman algorithm introduced in 2002 works on the basis of identiifying edges which link communities. In the same way that a node can have a betweenness coefficient so can an edge, finding the edges with the highest betweenness (or being critical choke points for connecting sets of nodes). To find communities we can iteratively remove edges from the network until we have found the number of distinct communities

Extendded into Clauset-Newman-Moore greedy algoirtmg for modularity maximisation we work on the basis that each node starts as it's own community. A pair of communtiies are identified which would produce the greatest gain in modularity for the overall network by combining them together. A benefit to this approach is that the algorithm will terminate when there is no more possible gain to modularity and so there is an inherent restriction on the number of communities.

There exists another method to find such communities, take the edge betweeness of each edge, find the highest edge betweeness and cut that edge, continue this until you have reached your desired number of communtiies. An inherent weakness of this approach is that the number of communities is difficult to determine, the distribituion of node degrees is hard to discern without computation and it varies wildly between iterations of increasing community numbers.

Depending upon network structure finding the number of communities which would be useful for working with influence models is a difficult problem.

3.7 Working with multiple metrics

TOPSIS - Seems to be a ranking of nodes based on placing each in a vector space based on some metrics, determining "optimal" for each metric and then choosing nodes that have the lowest distance from that optimal point in the space.

Is this good, bad? Time complexity, memory cost, size of constant factors?

Can it be easily approximated with just combining naive heuristics in a single factor. Compare to hybrid influence scores like s(v) = deg(v)*1/lcc(v)

3.8 Evaluating network topology

As a network is a graph made up of nodes and edges it's obvious to consider the number of nodes and edges or how these edges are distributed between the nodes. There are many metrics we can use to gain an insight into the type of network topology we are working with.

1. Diameter - The length of the longest shortest path from any two nodes in the network. Example uses are for example if a graph has a diameter of two then it could be a star graph

or if it has a diameter equal to the number of nodes then it must be a ring.

$$D(G) = \max_{v,u \in V} \{|shortestpath(u, v)|\} \tag{6}$$

2. Clustering Coefficient - The mean clustering coefficeint of each node in the network.

$$C = \frac{1}{n} \sum_{v \in V} \frac{2T(v)}{deg(v)(deg(v) - 1)} \tag{7}$$

where $T(v)$ denotes (TODO). This measures how closely clustered nodes are, it's a good measure of community structure with a high clustering coefficinet indicating that nodes can be split into distinct communtiies easily with a low coefficient indicating there is not much community structure.

3. Centralization - a measure in the range zero (0) to one (1) which determines how unequal the distribtion of node degrees is with a star newtork of the same size. A value of one indicates that the network is a star whereas a value of zero indicates that the each node has the exact same degree.

$$Cent(G) = \frac{\max_{v \in V}(\deg(v))}{n} - \frac{mean(\deg_{v \in V}(v))}{n - 1}$$

4. Mean degree - A very simple measure of the average degree of each node. High mean degree's show that a network is very dense and likewise low values indicate a sparse network. Small numbers of very high degree nodes do need to be considered as they will skew the average degree higher.

3.8.1 Approximations of metrics

Randomized approximation for betweenness centrality - How good is it. how much quicker is it. Is it worthwhile to use. Is betweenness a worthwhile measure

Discuss the approximations for local clustering - their runtimes, comparing to worst case of n cubed for high degree nodes, how much worse do these perform, is it worth doing, is n cubed too slow to be useful if more exact??

4 Problem Formulation

A network is defined as a graph $G = (V, E)$ where $|V| = n$ and $|E| = m$ The neighborhood of a node v is defined as $N(v) = \{(v, u)|u \in V\}$, this is the set of nodes which are connected to v. The two hop neighborhood of v is $N_2(v) = \cup_{u \in N(v)} N(u)$, this is the neigbourhood of each neighbor of v.

In the independent cascade model each node can have some state within $S = \{active, inactive, used\}$, each attempt for a node to influence it's neighbours as a probability p of succeeding, p is uniformily distributed across the range $[0, 1]$. Nodes may only change from "Active" to "Inactive" to "Used", we terminate once there are no more active nodes.

The decreasing cascade model is a further modification of the independent cascade model where each node's probability of being turned is computed before starting and on each attempt it's either accepted or put through a function which monotonically decreases it's value.

In the linear threshold model each node is given a threshold t which is uniformly distributed across the range $[0, 1]$. Once the proportion of v's neighborhood which has become infected is greater than or equal to t then the node v will also become infected.

$\sigma(s, t)$ is the set of shortest paths between s and t, likewise $\sigma(s, t|v)$ is the set of shortest paths between s and t which contain the node v.

$IM(G = (V, E), k)$ is the problem to find a subset of size k of V which will produce the greatest gain in influence after running a model of our choice on it.

5 Proposed Heuristics

We propose the following algorithm as a simple heuristic for choosing influential seeds. We give each node in the graph a score which is the ratio of the node's degree to the mean degree of that node's neighborhood.

```
def Deg-div-mnd(G: nx.DiGraph, n: int):
    def __neighbour_deg(G, node):
        neighbour_degrees = [G.degree(n) for n in G.adj[node]]
        if neighbour_degrees == []:
            return 0
        return sum(neighbour_degrees) / len(neighbour_degrees)

    score = []
    for node in G.nodes():
        score.append((G.degree(node) / (1e-6 + __neighbour_deg(G, node)), node))
    score.sort(reverse=True)
    return [x[1] for x in score[:n]]
```

The rationale behind this combination is that high degree nodes are well known to be highly influential across a wide variety of networks. An problem with a degree based approach is choosing nodes which are clustered together making the the gain in influence lesser as the number of seeds increases.

Dividing by the mean degree of the node's neighbourhood means that we are targeting nodes which are connected to a large amount of poorly connected low degree nodes. We hypothesis that this will produce a much better set of seeds than the standard degree heuristics particularly as the number of seeds increases.

5.1 Methods which did not yield viable results

From experimental testing it is clear there are differences in how certain methods for finding seeds perform based on the number of seeds required in relation to each-other. Some methods such as degree and degree discount based approaches work well at low seed counts but demonstrate increasingly diminishing returns as the number of seeds increases. This behaviour is expected due to the sub-modular nature of our problem space however other methods demonstrate that it may not necessarily be the case for all heuristics.
The following are the ideas and reasoning behind them, which produced interesting results across at least one of the network topologies and methods.

1. Custom 1: $S(k) = \deg(\log(k)) \cup bet(k - \log(k))$. Running trials of both betweenness and degree across multiple networks and then computing the intersection of the two approaches terminating states indicated minimal intersection of the two methods. When combined in this manner performance was good on the Enron network, on this network clustering coefficient and betweeness were particularly good indicators of spread. On the other tested networks this did not perform well at all giving very low performance.

2. Custom 3 $s(v) = \deg(v) \times (1 - lcc(v))$. This will find nodes that are a combination of high degree and low clustering coefficient at the same time. Nodes with a low clustering coefficient are relatively disconnected to the clique that would be formed by their neighbours, this would be an excellent way to find nodes which may have the greatest reach through the network in terms of distance and therefore distribution rather than immediate raw node counts.

3. Custom 4 $s(v) = \deg(v) \times \frac{1}{lcc(v)}$. Going to the reciprocal of clustering coefficient will weight the choices more towards the lower clustering nodes than that of degree as in Custom 3.

4. Custom 6 $s(v) = \deg(v) \times (1 - \frac{1}{max_{u \in V}(mnd(u))} mnd(v))$. Following on from Custom 5 we attempt to choose low mean neighbour degree nodes with a larger focus on node's degree. By

normalising mean neighbour degree into an interval $[0, 1]$ we aim to improve the performance of Custom 5 at lower seed counts whilst maintaining expected performance.

5. Custom 7: $s(v) = \deg(v) \times (1 - lcc(v)) \times mnd(v)$. The same logic as with taking high degree low clustering nodes but weighting for a higher mean neighbour degree. We aim to choose nodes which have lots of far reaching connections due to the low clustering coefficient, each of those connections has a high number of connections.

6. Custom 8: $s(v) = \deg(v) \times bet(v)$. Taking nodes with a high degree and a high betweeness at the same time will skew towards influential (due to degree) bridge nodes between relatively disjoint parts of the network

Note that $s(v)$ denotes a score for each node, to choose k nodes simply choose the k nodes with the highest score. $S(k)$ denotes the set of k nodes made up of different sets.

Testing will be done using MHDA samples of each of the four networks with one thousand (1000) nodes, this is lower than the smallest size of our datasets. it will also provide a good comparison to most papers which use networks of around this size. At this scale performance of algorithms such as betweeneesss centrality is more than acceptable. Using larger sizes would require dedicating significant resources to the problem, using approximations for some metrics or using completely different methods that do not rely on these metrics. The naive greedy algorithm would not be feasibly to run on larger networks.

6 Experimental Method and Setup

This section describes the testing methodology used to conduct experiments, networks used and their properties, and any statistical methods. This is to ensure that each result described can be reproduced with the methods and reasoning clear and simple to understand.

6.1 Datasets

I have used the following real world datasets in my testing. As shown in the table below **??** they range in sizes from just over fifteen hundred (1500) nodes to just under a hundred and seventy thousand (168114) with between seventeen thousand edges (17215) up to over six million and seven hundred thousand (6797557).

Name	Nodes	Edges	Mean Deg.	Cluster Coef.	Diam.	Centralization
Enron	36692	183831	10.020	0.497	13	0.00725
P2P	10876	39994	7.355	0.006	10	0.001899
Airports	1574	17215	21.874	0.504	8	0.004334
Twitter	81306	1342310	33.019	0.565	7	0.002114
Physics	18772	198110	21.107	0.631	14	0.001328
Twitch	168114	6797557	80.868	0.160	8	0.004709

Figure 1: Our network metrics

1. Enron. Enron was an energy company which went bankrupt in 2001, there was a large amount of fraud and shady practices, as part of the investigation their internal emails were released. These emails form a large network with large

2. P2P represents a snapshot of the Gnutella peer to peer network from August 4th 2002, an it has a very interesting topology where nodes are split into peers or leaf nodes and "ultra peers" or hubs. The leaf nodes are connected to up to 5 other nodes. Each of the hub nodes has approximately 32 connections to both leaf nodes and other hub nodes. For a network of this size it has a relatively high diameter due to it's almost grid like nature with overall poor connectivity across it. Targeting those hub nodes which are primarily connected to the leaf nodes rather than other hub nodes will likely yield a performant option. [**?**]

3. Airports is a reperesentation of the air traffic across the top 500 American airports by passenger volume observed in 2008 (TODO), we note that there are over 1500 nodes when we are considering 500 airports. This is due to American airports having many flights to international airports. These international airports are likely to act in a similar manner to the leaf nodes in the P2P network. Nodes are highly clustered in this network which is evident from the typical hub and spoke model employed by airlines.

4. Twitter represents a small subset of it's user base in (TODO) and their circles, this is no longer a feature and was removed in 2023. This allowed people to select a subset of their followers for which certain tweets could be shared to rather than everyone that they were followed by. This dataset exhibts a relaively low centralisation, a high mean degree as well as a low diameter and is an excellent represenation of a social network in a non work context.

5. Physics dataset is all academic paper citations between 1993 and April 2003 within astrophyscics papers on arXiv. It is an example of a professional network displaying a high mean degree, very high clustering but poor centralization. This is easily explained by certain topics having lots of closely cited papers with there not being a very large amount of overlap between these different groups.

6. Twitch is an interesting dataset. It is our largest dataset and represents the

6.2 Implementation of Greedy and CELF

Some heuristics have very similar performance across different models of spread such as degree. Some methods such as betweenness or clustering coefficient have incredibly varied performance across the different diffusion models. As both methods require running a cascade on the network, the corresponding diffusion model is chosen by the function and any marginal gains in influence are done using the required model.

We did not use greedy in any of the comparisons as CELF has the exact same approximation guarantee and is feasible to run over our networks. When computing marginal gains it is typically done by using a monte carlo simulation of repeatedly rerunning the cascade with each seed set and candidate node to find an average. Due to the very large computational cost associated with performing monte carlo simulations across multiple networks we check how different our simulations are using a Mann-Whitney U test.

Computing the observed effect size using Cohen's d for the two methods gives values in the range 0.206 to 0.238 for all seed counts except 100, with 100 having a value of 0.37 showing that the observed effect size is small. Our standard deviations are small ranging from under 0.02 to 0.034, with our largest observed effect the difference between the two distributions is 0.37 of a standard deviation which for the 100 node selection is equivalent to them varying by 0.74 percent in end coverage. To be able to detect such a small effect size with a power of 80 percent we would need to perform between 115 and 372 samples. Running just thirty with the monte carlo method takes over 90 minutes in parallel on the compute cluster. For our purposes the small effect size, minimal standard deviations, large p values and the required computational power show that the two are able to be used interchangeably particularly if we are able to use larger numbers of trials using the more efficient method.

Seed Size	Mean Cov. CELF	Mean Cov. CELF with MC	U-Stat	P-Value	Same Dist.
10	0.378918	0.371873	521.0	0.297272	Yes
50	0.399411	0.404740	416.0	0.620394	Yes
100	0.421773	0.414289	530.0	0.239843	Yes
200	0.440918	0.434511	524.5	0.273927	Yes
300	0.461211	0.457658	500.5	0.459754	Yes

Table 1: Mann-Whitney U test $\alpha = 0.05$ on the Monte Carlo CELF (100 simulations) and Non Monte Carlo CELF method over 30 runs on the Enron dataset sampled using MHDA with 1000 nodes

6.3 Sampling Methods

1. BFS (Breadth First Search). Have a very significant tendancy to lead towards high degree nodes leading to globally non representative samples. We won't use BFS for general sampling however it does have uses for algorithms which may only required a localised view of the network from a certain point.

2. MHDA (Metropolis Hastings Random Walk with Delayed Acceptance). As described in the literature review this is a performant and representative sampling method. We will use this for all samples when comparing the performance of heursitics as it generates represnetative samples as well as being commonly used for this purpose in various papers (TODO CITE).

6.4 Accounting for differences in network sampling

Performing analysis on a single (non deterministic) sample of a network will not create reproducible results. This is due to not producing a representative sample of the network by each sample having variances in topology, density, connectivity and therefore how well certain heuristics may work.

As most heuristics are deterministic in which seeds they will return for a given network and required size, running the same method on the same network is useful for finding out the noise from the diffusion model rather than variances between different methods.

To perform any statistical analysis of say k runs I will sample the network k times and then each method will be run on each of these networks. This guarantees that the network is covered by each of the samples and will allow us to generalise to the whole network.

6.5 Evaluation Methods

Measuring Final Spread, Rate of Spread, Variability due to changes in parameters, Runtime. All metrics will have a mean, standard dev computed into a tabular format to allow for very quick evaluation of statistical significance.

6.6 Baseline Behaviours

Degree is a very good indicator of influence spread peformance. Single and double degree discount are also excellent.

CELF performs worse than greedy in majority of circumstances, whilst being over 100 times faster.

TOOD talk about why the behaviour shown in the above figures is - highlight the randomness between run to run, talk about cumulative effect of choosing algorithms for a specific network being highly flawed by selected a super high performing heuristic+sample combination on one net that performs very poorly on another.

6.7 Statistical Analysis

There are a few ways to compare our results, it is important that the analysis is rigorous and shows clear statistical significance.

We are comparing the performance of a method across networks, we need to take into account usage of samples within those networks and random variability between runs of each algorithm on each sample. The number of unique samples is very important to have a number high enough. If we had 20 runs per sample with 5 samples of a network it might seem that we have 100 data points to compare. This is not the case, we know that network topology is slightly different between samples and network topology has a large effect on the results of a heuristics performance, each of the sets of 20 runs are correlated with eachother and so cannot be treated as independent trials. Due to this I will use a linear mixed effects model, for each run of a heuristic I will

6.7.1 The data is a curve

A plot of influence spread against starting influential seed set would produce a curve in a similar shape to a logarithm due to the sub-modular nature of the spread function. There are two strategies we could use to compare two methods

1. Use the area under the curve. This might seem as the best way of comparing the performance of two methods however it is a very coarse grained approach and will not allow for analysis of performance at different seed size proportions. Methods which perform very well at low seed counts but show minimal improvements as k increases could look very good when in actuality they are not something that you would want to use due to variance.

2. Compare the heuristics at a selection of certain points. Different heuristics perform better or worse at different starting seed counts on different networks, having the most precise data at each point as a proportion of the overall network is a better way to make sure we choose the correct method for our specific usecase.

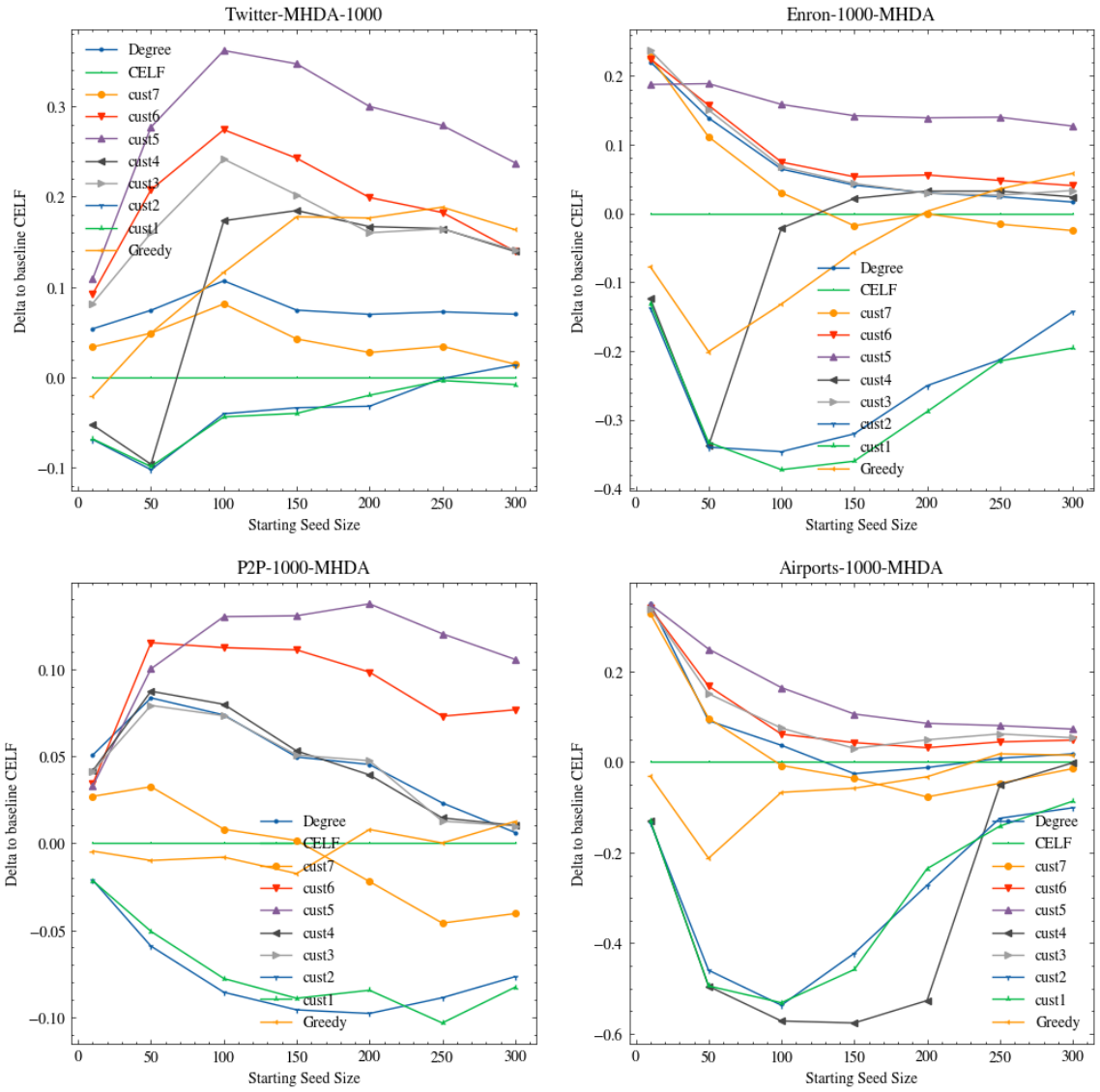


Figure 2: Baseline Performance overall all models on all networks

6.8 A note on performance

Even on small 1000 node networks there are large performance differences between heuristics and methods. These runtimes include the time taken to initialise a new worker thread, read and unpickle the network to be operated on from disk, execute the heurisitc function, use the seeds on the given network and then save results to a new file relinuqishing our worker thread back into the pool. This overhead is most easily seen with the difference between Degree and Deg-div-mnd being only a few miliseconds despite performing up to a factor of $O(\Delta^2)$ more operations where Δ is the maximal degree of a node.

Method	**Mean Runtime**	**Std. Dev Runtime**	**Big O average case**
CELF (Monte Carlo)	9.14×10^5ms	4.61×10^4ms	$O(100nm + k \log m)$
CELF	1.86×10^4ms	1.04×10^3ms	$O(nm + k \log m)$
Degree	26ms	2.3ms	$O(m + n \log n)$
Local Clustering Coef.	153ms	102ms	$O((\frac{m}{n})^3)$
Betweenness	348ms	166ms	$O(nm)$
Deg-div-mnd	33.6ms	6.3ms	$O(m + n \log n)$

Table 2: Runtime statistics for choosing 300 seed nodes on samples of 1000 node samples Enron under the MHDA sampling algorithm over 30 trials using the IC model with $p = 0.2$. The monte carlo CELF used 100 simulations showing minimal overhead to the test script.

CELF is around 700 times slower than degree and degree-div-mnd based methods. CELF is also around 700 times faster than the un-optimised greedy algorithm as shown by [**?**]. Both the clustering and betweeness based methods show they are between five to ten (5-10) times slower than basic degree based methods. Clustering depends upon how dense or sparse the network is, if it was a complete graph for example then m would be equal to n^2 and clustering approaches would be cubic and offer the same performance as betweeness. Any approaches using either of these would therefore have a significantly larger runtime than that of simpler heurisitcs.

6.9 Tools and Libraries

For the data visualisation the library Matplotlib [**?**] was used for plotting along with Scienceplots [**?**] for creating consistent formatting and fonts across all figures. The following libraries were used in the code and statistical analysis: Numpy [**?**] for general mathematical functions, Pandas [**?**] for data storage, saving and reading files, Scipy [**?**] for the statistical analysis, Networkx [**?**] to allow for the implementation of graph based methods and models.

7 Results and Analysis

In this section we discuss the strategies proposed previously, how they perform across our range of networks, was this behaviour as expected and were there any outliers. We will also analyse each of the methods to test for statistical significance compared to known methods.

7.1 Hybrid Algorithms

7.1.1 Parameters

1. Networks: Each of the networks was sampled using the MHDA algorithm to produce a unique network for each trial. Each of the different heurisitcs used the same set of networks for consistency and to limit the random variance from both heurisitc and network topology.

2. Baseline Comparisons: Using CELF and degree as base heuristics to compare against the new hybrid heuristics. There are few heurisitcs which perform well and are efficient to calculate for so many networks.

3. Seed Sizes: Each sampled network is made up of one thousand (1000) nodes, when comparing methods the seed sizes to compare will be from 25 to 300 in 25 node incremements with the addition of 10 nodes to represent incredibly small starting seed proportions.

7.1.2 ICC Analysis

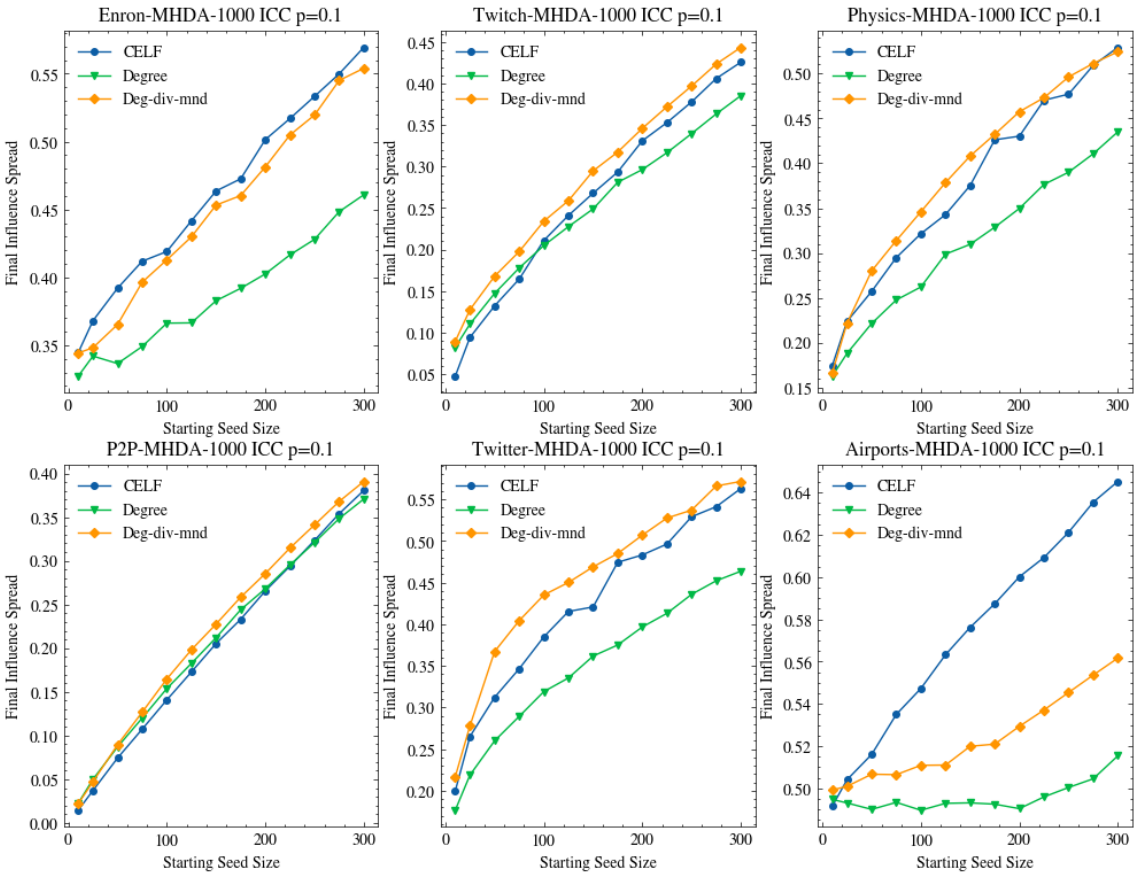


Figure 3: ICC Performance $p = 0.1$ $n = 100$

As shown in figure **??** the IC model with $p = 0.1$ each of our six networks we outperform or at least match the degree heuristic across all seed sizes. In all networks except the Airports we see

our heuristic performing very close to CELF and even exceeding in some cases.
We hypothesize that the large difference in performance on the airports dataset (TODO)

7.1.3 LTM Analysis

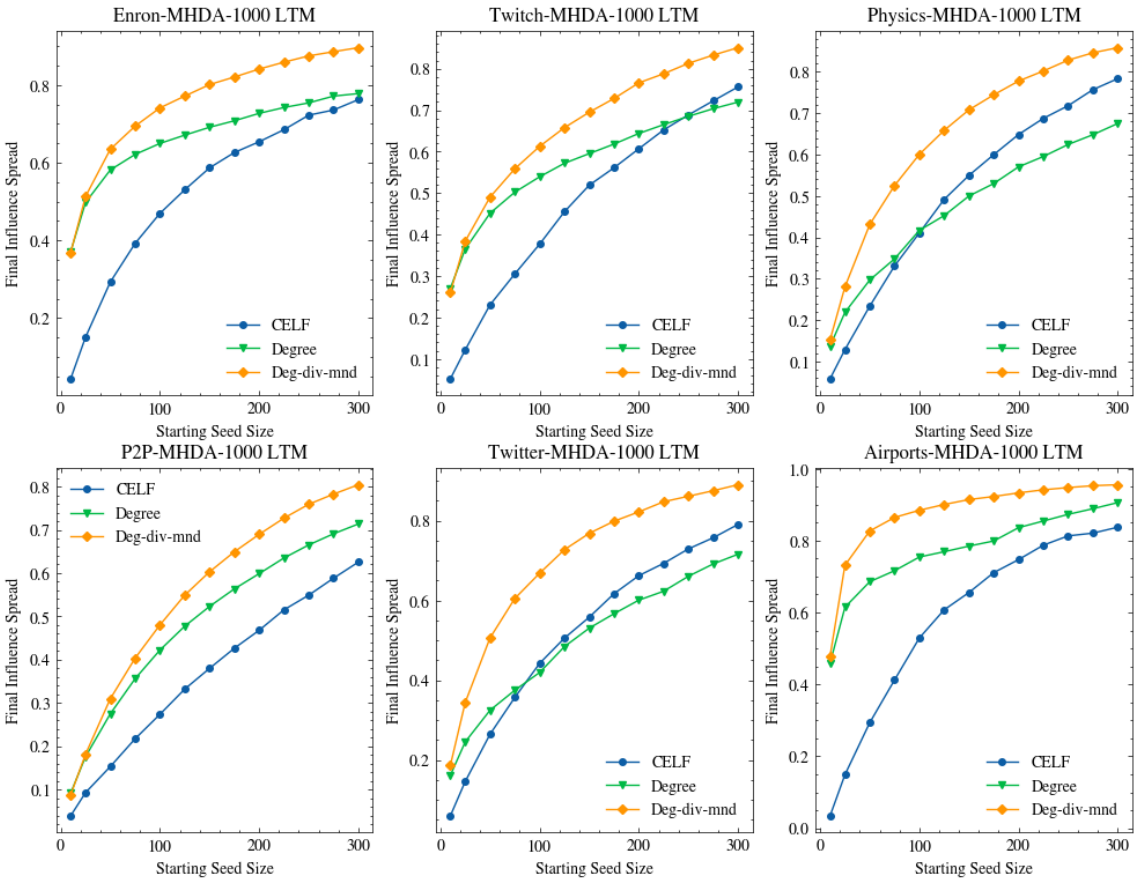


Figure 4: LTM Hybrid Heuristics Performance $n = 500$

As shown in figure **??** our heuristic shows a clear performance improvement over both degree and CELF across all seed set sizes.

We note that the variance in performance from CELF and our degree based methods is due to the way CELF will compute the marginal gain for each node using just that node. For the threshold model computing an expected gain for a node is almost random as the full effect of the threshold of neighbours being affected does not take effect.

7.1.4 DCC Analysis

As seen in figure **??** our heuristic shows superior performance to both degree and CELF across the entire range of seed sizes and networks.

7.1.5 Stability across model parameters

It is not useful to find a heursitic which only works across a single narrow range of model parameters, in our case $p = 0.1$ and $q = 0.7$. We have therefore tested each of the IC and DC models with a set of parameters as listed in table **??**

TODO EXPLAIN WHY

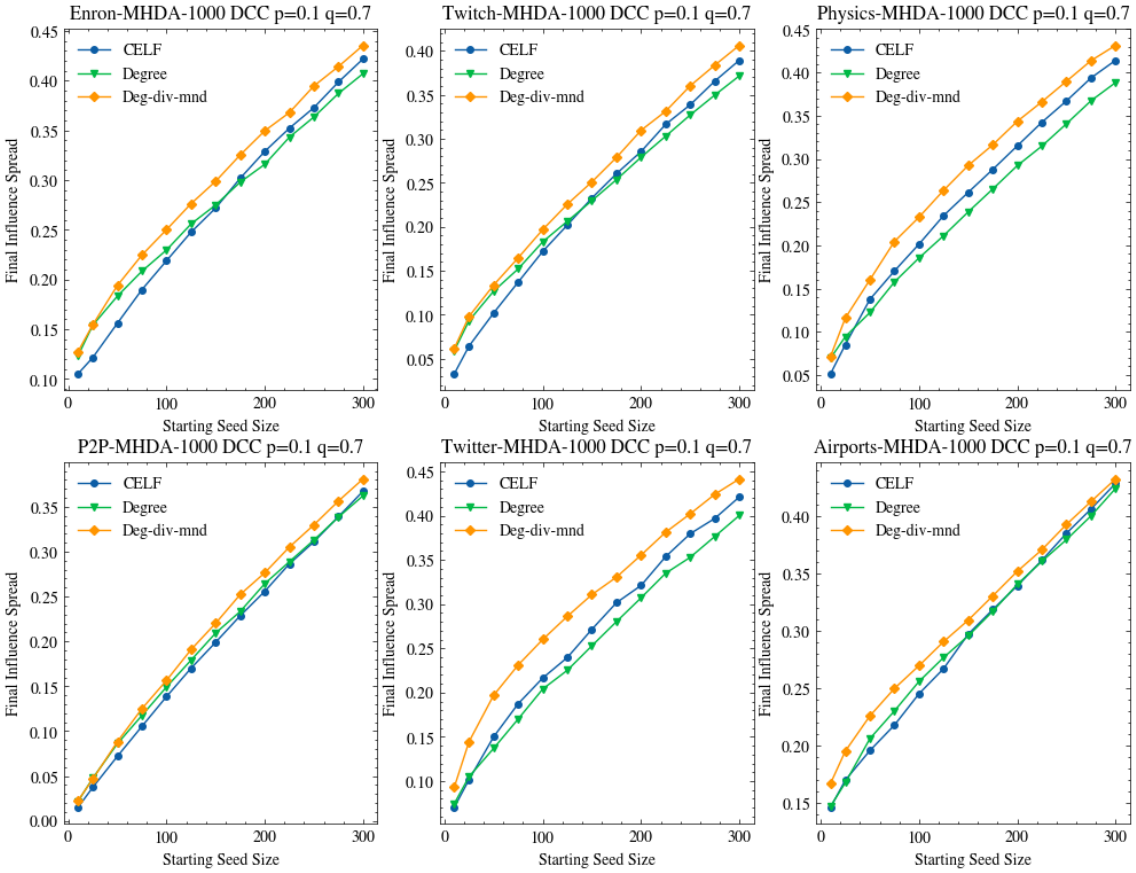


Figure 5: DCC Hybrid Heuristics Performance $p = 0.1$, $q = 0.7$, $n = 100$

7.1.6 Stability of the IC model

7.1.7 Stability of Decreasing Cascde

As shown above the heursitic is stable across varied values of p under the IC model. We test now for variance when under the value of q in the recurrence relation between the probability of activation over time $p_{t+1} = q \times p_t$.

7.1.8 Stability of the threshold model

Rather than assigning a random threshold for each node in the range [0, 1] we can also give all nodes a fixed probability. Setting a threshold θ of 0.5 should produce a very similar output to that of the randomised thresholds over the range 0 to 1 as the expectation of a random variable over that interval is 0.5.

Model	p	q	t	Reasoning
IC	0.1			Our Baseline
IC	0.2			More likely to spread
IC	0.3			Even more likely to spread
DC	0.1	0.5		
DC	0.1	0.6		
DC	0.1	0.7		Our Baseline
DC	0.1	0.8		
DC	0.1	0.9		Expect to trend close to IC
LTM			0.9	Expect the gap to open between the methods
LTM			0.7	Expect slower growth and lower overall values
LTM			0.5	Expected to be almost identical to baseline
LTM			0.3	Expect sharper growth and higher overall values

Table 3: Tested Parameters for Regression

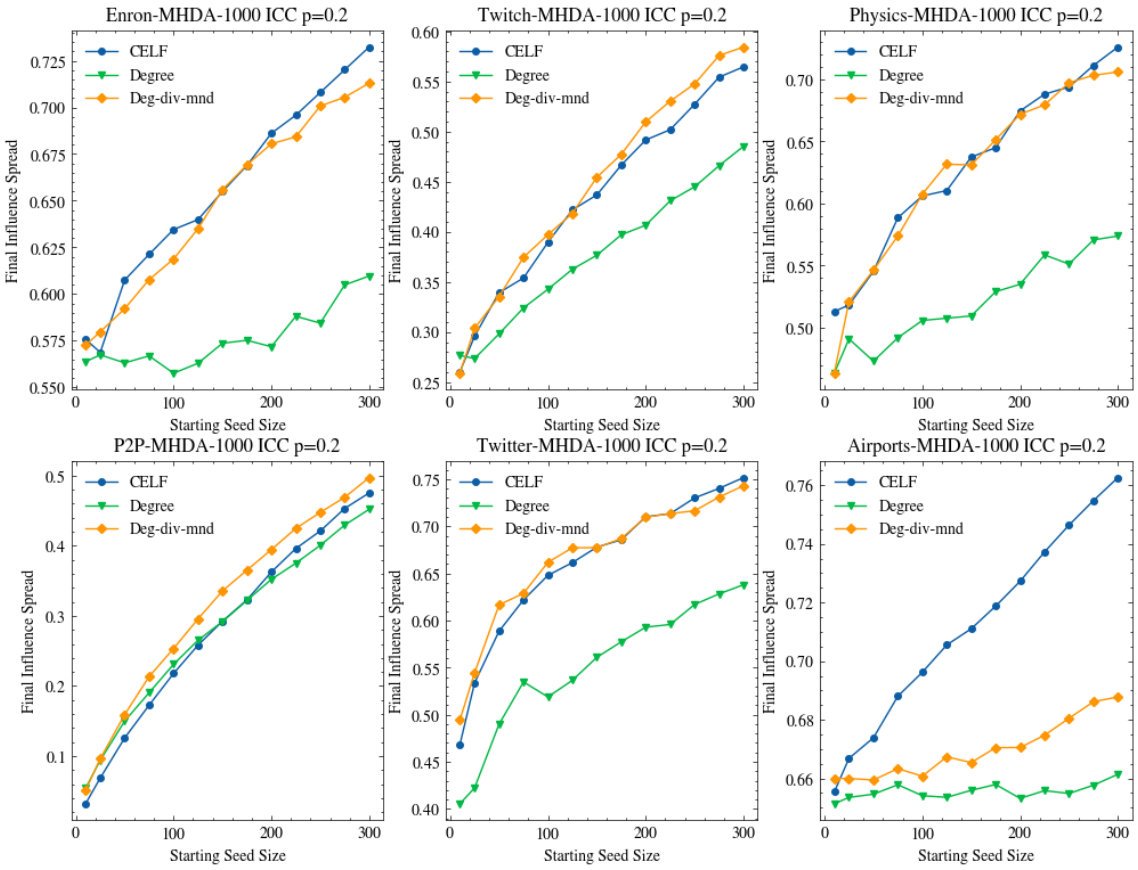


Figure 6: IC Stability at $p = 0.2$

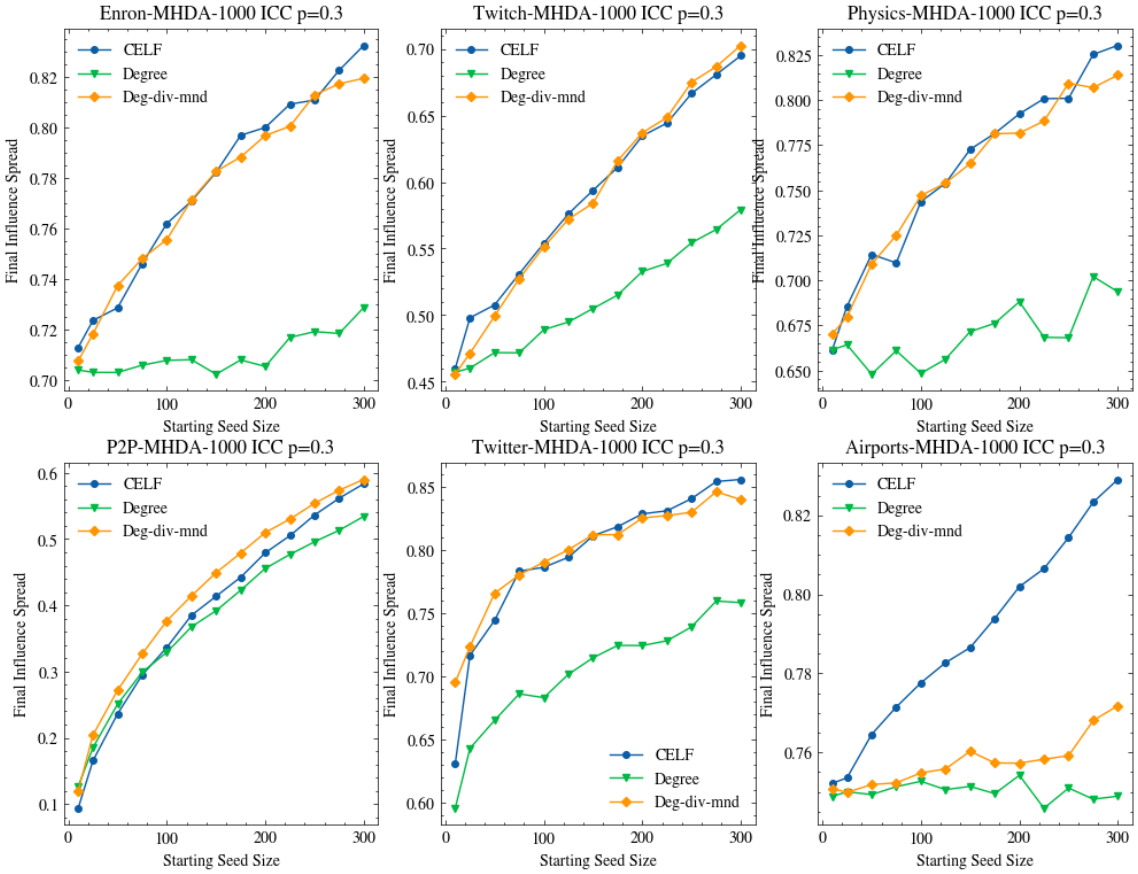


Figure 7: IC Stability at $p = 0.3$

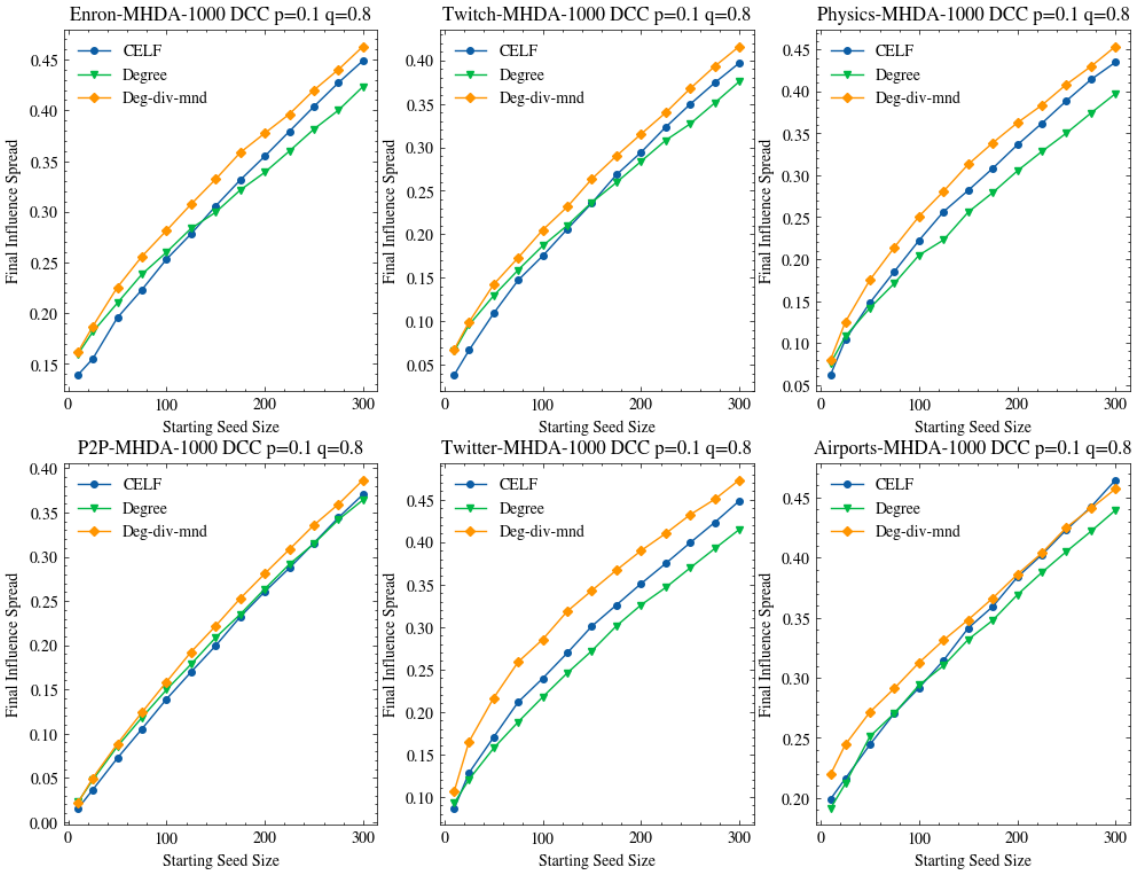


Figure 8: DCC stability at $p = 0.1$ $q = 0.8$

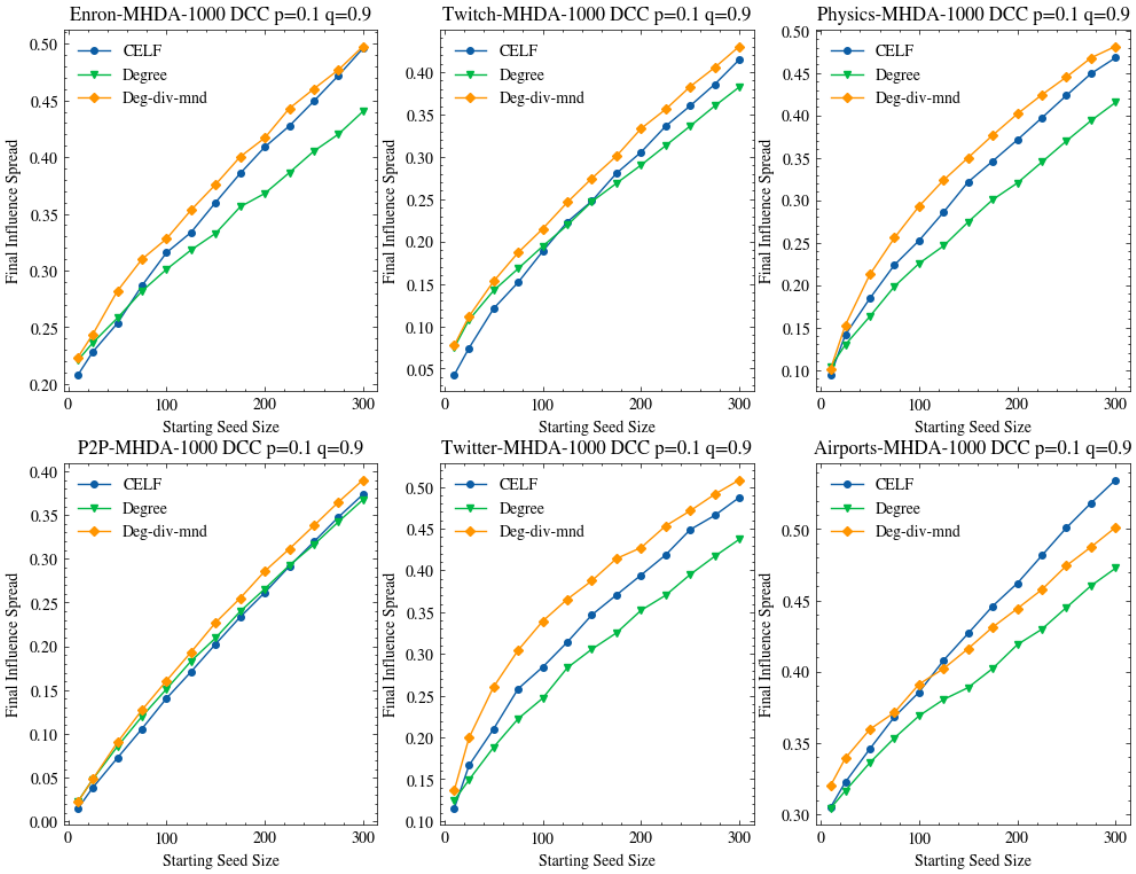


Figure 9: DCC $p = 0.1$ $q = 0.5$

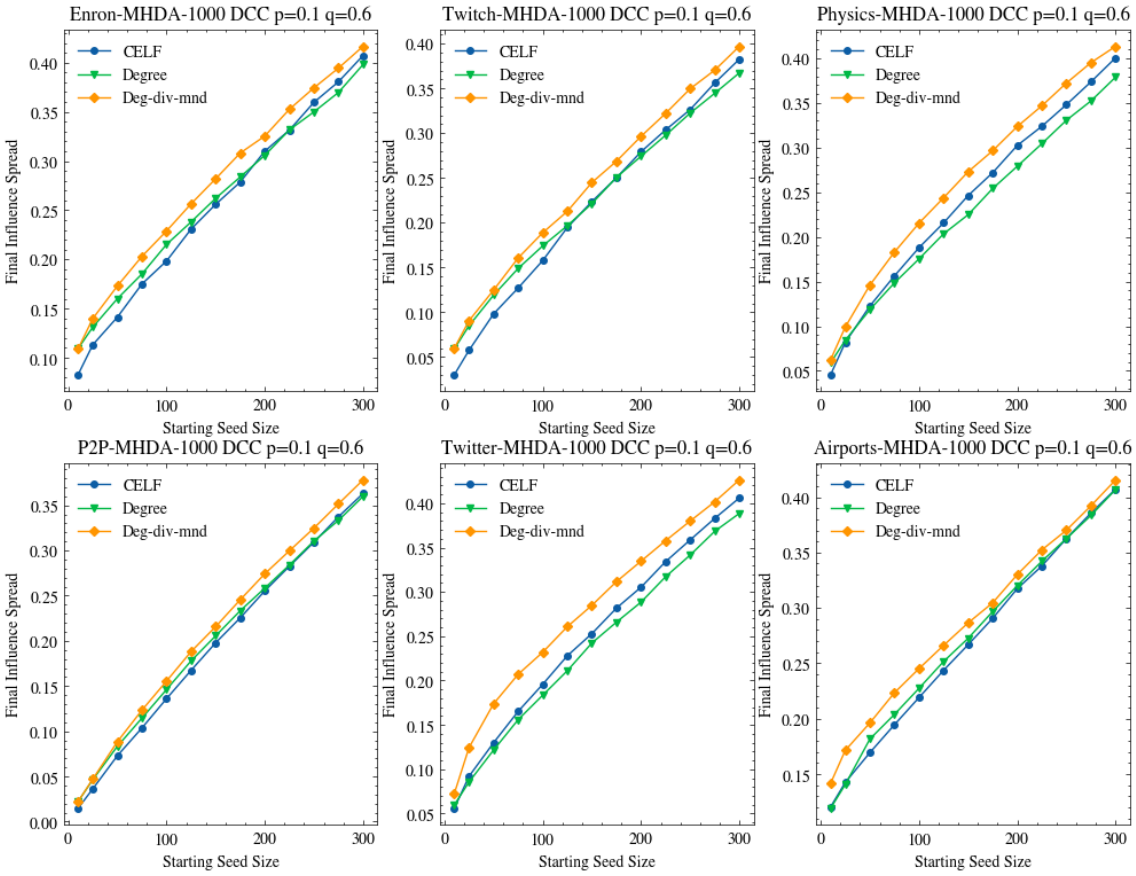


Figure 10: DCC stability at $p = 0.1$ $q = 0.6$

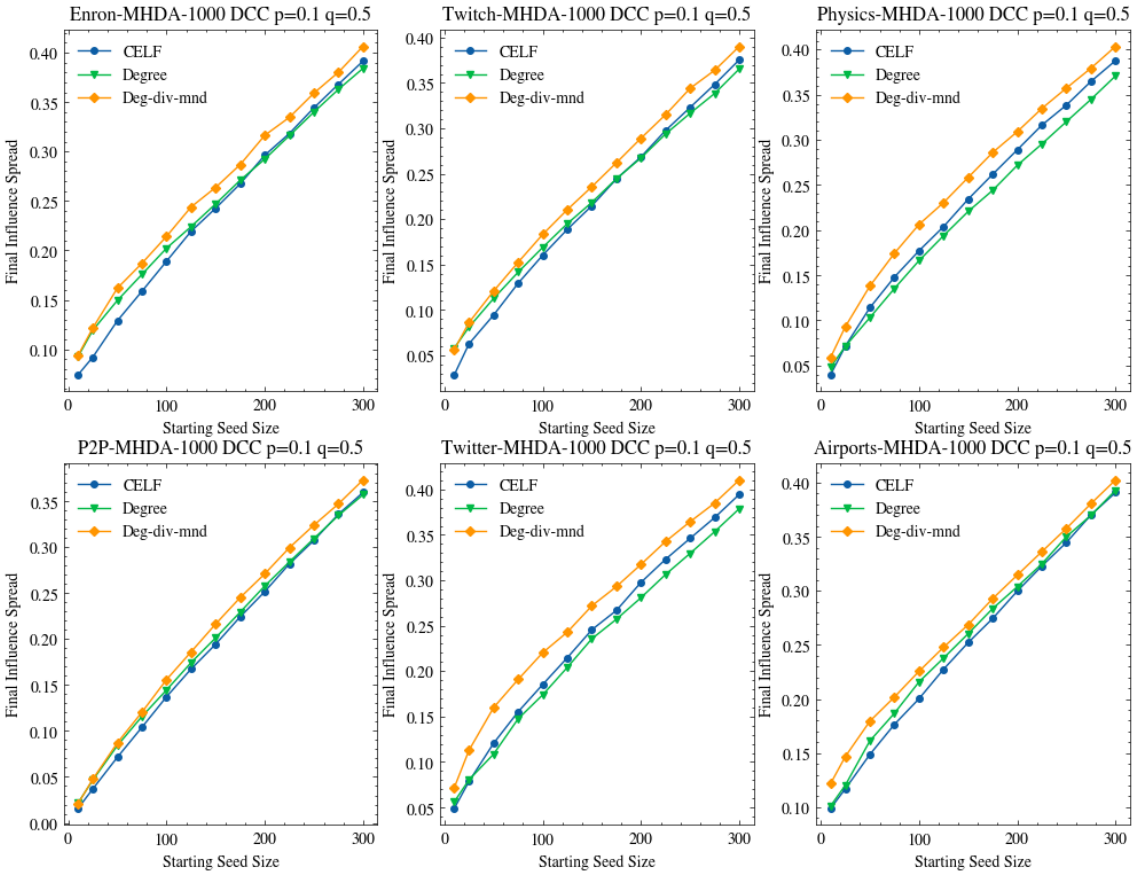


Figure 11: DCC $p = 0.1$ $q = 0.5$

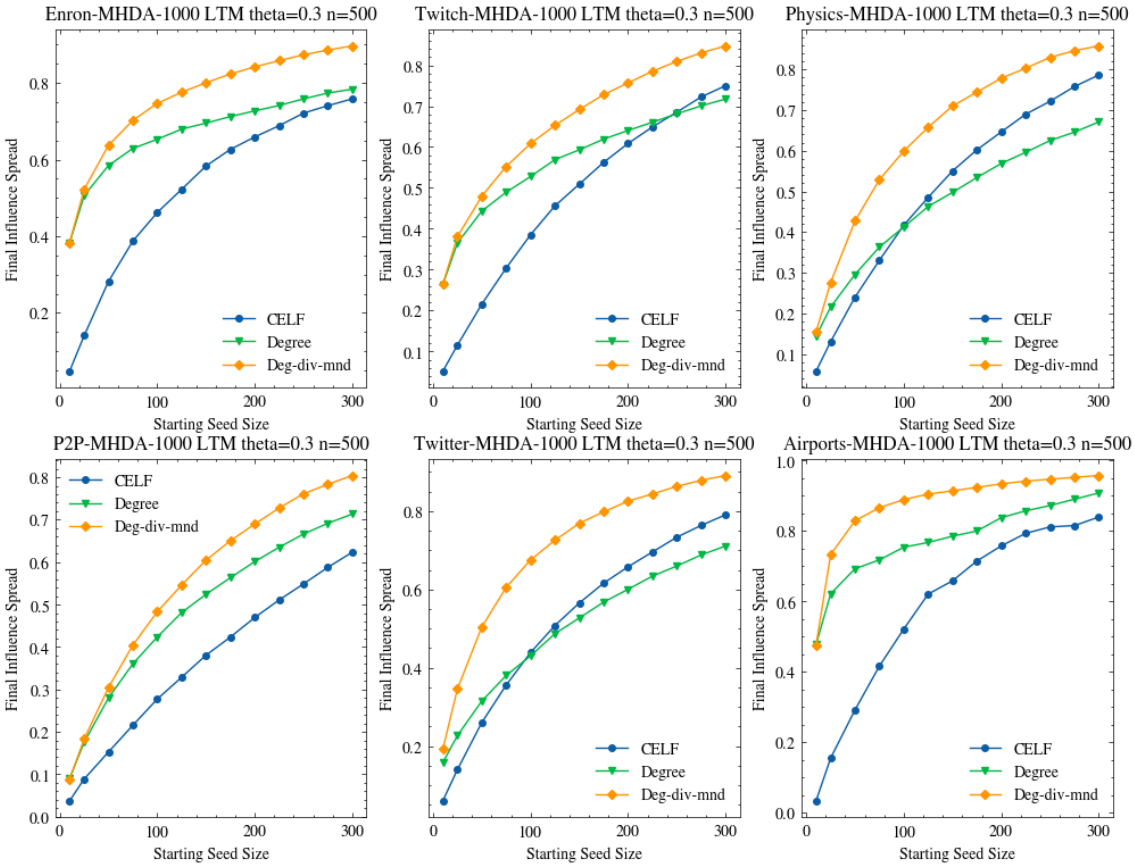


Figure 12: LTM stability at $\theta = 0.3$

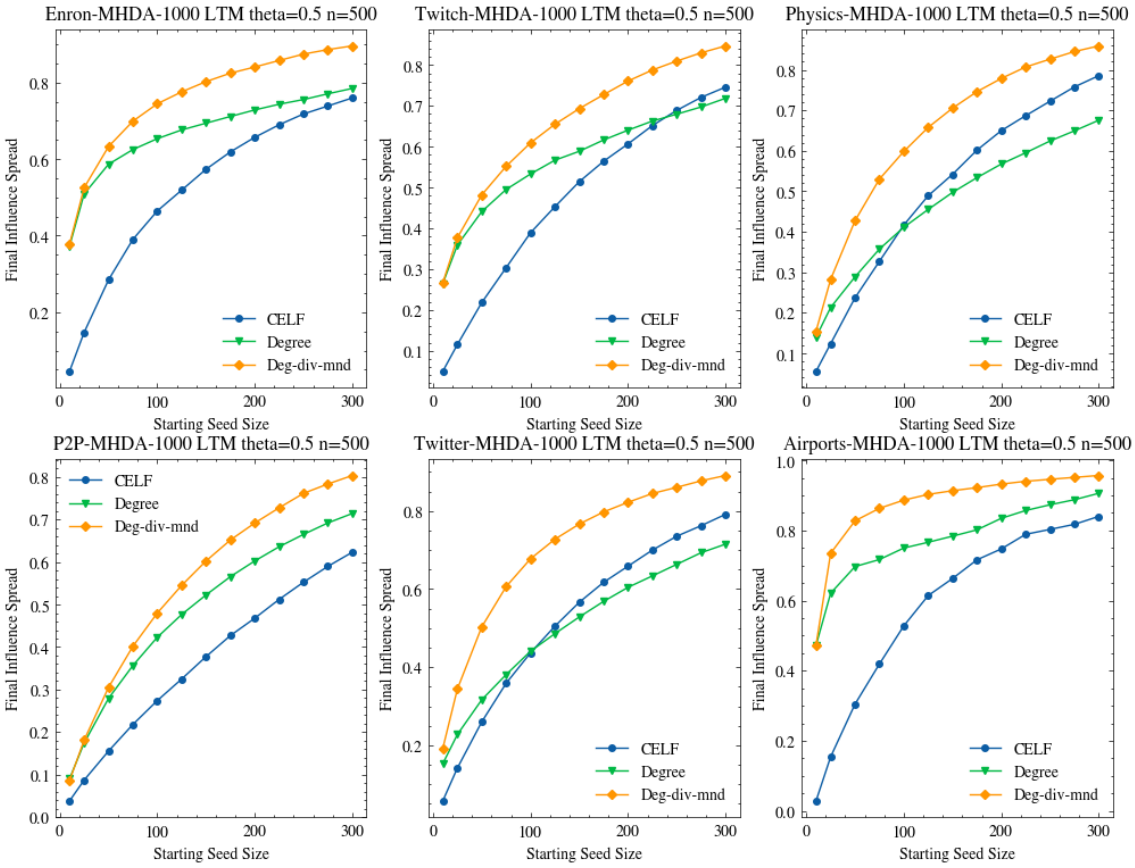


Figure 13: LTM stability at $\theta = 0.5$

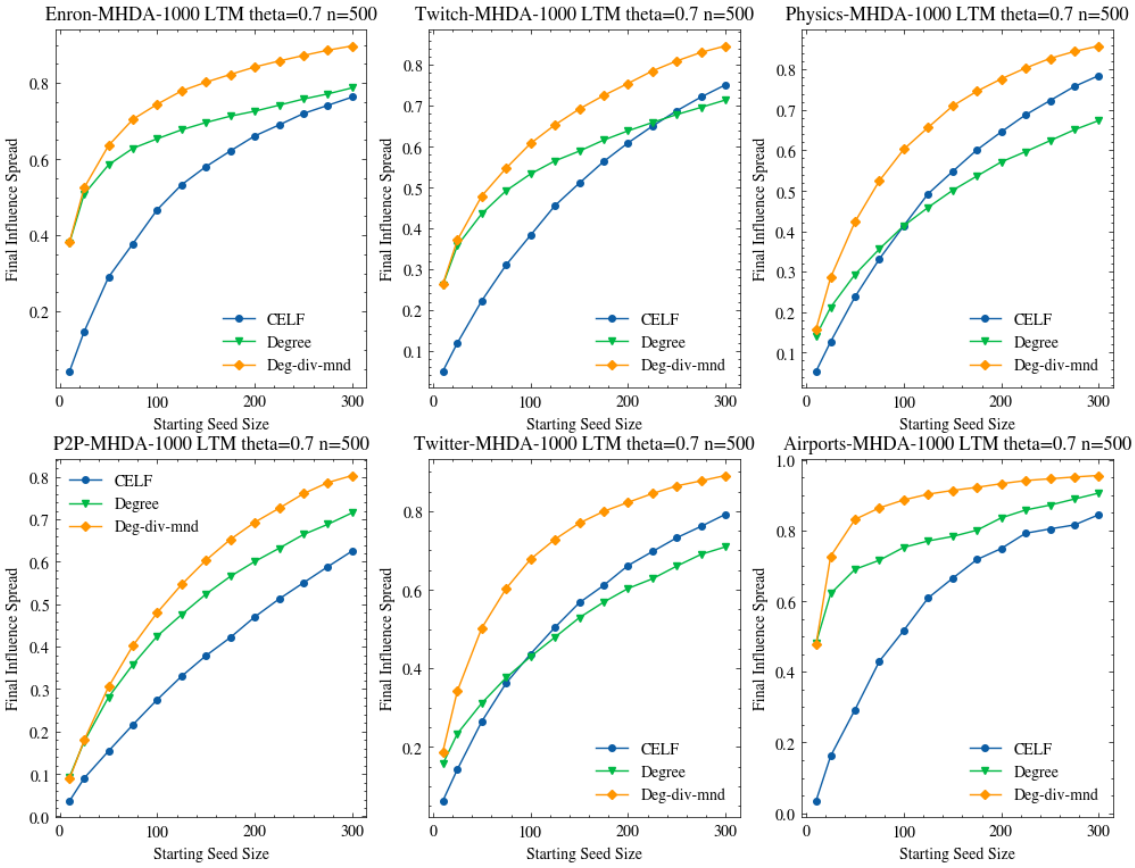


Figure 14: LTM stability at $\theta = 0.7$

7.1.9 Methods which did not work

1. Solely using betweenness or local clustering coefficient of a node as an indicator of influence was very network dependant and barely exceeds degree on most topologies.

2. Attempting to improve custom 5 (in Custom 9) by means of choosing $logk$ nodes as degree to improve the low seed count performance did not work and made it perform worse across all areas.

3. Union based approaches of selecting

4.

8 Limitations

We consider a wide range of networks that cover a large number of real world topologies. Using exact networks to compare with specific papers would be a good idea.

Investigation into the correlation between the relationship with seeds chosen by CELF in an IC or DC scenario and how they translate into influence under the LTM model would be a very interesting area to explore.

9 Conclusion

We have shown that for the influence maximisation problem a simple structural heuristics with no global knowledge of the network is able to outperform the greedy (1-1/e) approximation on the majority of networks and topologies.

The heurisitcs' behavior is consistent across a range of model parameters including varied activation probabilities, varied decreasing function slopes as well as both randomised and fixed thresholds under the threshold models.

References

[1] James Archbold, Sophie Clohessy, Deshani Herath, Nathan Griffiths, and Oyinlola Oyebode. An agent-based model of the spread of behavioural risk-factors for cardiovascular disease in city-scale populations. *PLOS ONE*, 19(5):1–22, 05 2024.

[2] James Archbold and Nathan Griffiths. Limiting concept spread in environments with interacting concepts. In *16th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2017)*, pages 1332–1340. IFAAMAS, 2017.

[3] Henry Franks, Nathan Griffiths, and Sarabjot Singh Anand. Learning agent influence in mas with complex social networks. *Autonomous Agents and Multi-Agent Systems*, 28:836–866, 2014.

[4] John D. Garrett. garrettj403/SciencePlots. September 2021.

[5] Minas Gjoka, Maciej Kurant, Carter Butts, and Athina Markopoulou. Walking in facebook: A case study of unbiased sampling of osns. pages 1 – 9, 04 2010.

[6] Minas Gjoka, Maciej Kurant, Carter T. Butts, and Athina Markopoulou. Practical recommendations on crawling online social networks. *IEEE Journal on Selected Areas in Communications*, 29(9):1872–1892, 2011.

[7] Sanjeev Goyal and Michael Kearns. Competitive contagion in networks, 2012.

[8] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using networkx. In Gaël Varoquaux, Travis Vaught, and Jarrod Millman, editors, *Proceedings of the 7th Python in Science Conference*, pages 11 – 15, Pasadena, CA USA, 2008.

[9] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020.

[10] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007.

[11] David Kempe, Jon Kleinberg, and Éva Tardos. Maximizing the spread of influence through a social network. In *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '03, page 137–146, New York, NY, USA, 2003. Association for Computing Machinery.

[12] David Kempe, Jon Kleinberg, and Éva Tardos. Influential nodes in a diffusion model for social networks. In *international colloquium on automata, languages, and programming*, pages 1127–1138. Springer, 2005.

[13] Tor Klingberg and Raphael Manfredi. Gnutella 0.6 release memo, June 2002. https://rfc-gnutella.sourceforge.net/src/rfc-0_6-draft.html Visited on 08/04/2026.

[14] Marcelo N Kuperman. Invited review: Epidemics on social networks. *Papers in physics*, 5(1):0–0, 2013.

[15] Jure Leskovec, Andreas Krause, Carlos Guestrin, Christos Faloutsos, Jeanne VanBriesen, and Natalie Glance. Cost-effective outbreak detection in networks. In *Proceedings of the 13th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '07, page 420–429, New York, NY, USA, 2007. Association for Computing Machinery.

[16] The pandas development team. pandas-dev/pandas: Pandas, February 2020.

[17] Vedran Sekara, Arkadiusz Stopczynski, and Sune Lehmann. Fundamental structures of dynamic social networks. *Proceedings of the national academy of sciences*, 113(36):9977–9982, 2016.

[18] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17:261–272, 2020.

[19] Tianyi Wang, Yang Chen, Zengbin Zhang, Tianyin Xu, Long Jin, Pan Hui, Beixing Deng, and Xing Li. Understanding graph sampling algorithms for social network analysis. *2011 31st International Conference on Distributed Computing Systems Workshops*, pages 123–128, 2011.